ASSIGNMENT 1

CO1 – DSA0103

OBJECT-ORIENTED PROGRAMMING WITH C++

City Metro Smart-Card Fare Engine

Submitted by: Poojashri RS

Register Number: 192324266

1. Introduction and Assumptions

A city metro operator wants a fare engine that begins at one station (about 20,000 journeys a day) and grows to the whole network (about 3,000,000 journeys a day). At that scale four defects cause real money to be lost or disputed: **rounding errors** in fare arithmetic, **ambiguous peak-hour boundaries**, **balances that drop below the allowed minimum**, and **fare numbers copied into many places** so that one revision is applied inconsistently.

This report analyses the problem, designs a set of C++ classes, and then proves the design with programs that were actually compiled (g++ 13.3, `-std=c++17 -Wall -Wextra`) and executed. Every console output shown in the report is a genuine run, not a hand-written expectation.

1.1 Business rules and the interpretation used

Rule	Interpretation adopted in this report
Zones	Inner = Rs 10, Middle = Rs 20, Outer = Rs 30 (Airport = Rs 60 is added in Task 12).
Peak surcharge	20% of the base fare when 08:00 ≤ time < 10:00 or 17:00 ≤ time < 20:00.
Boundaries	08:00 and 17:00 are peak (start inclusive). 10:00 and 20:00 are off-peak (end exclusive).
Minimum balance	A journey is allowed only if balance − fare ≥ Rs 50.00 (so exactly Rs 50.00 left is allowed).
Money type	Integer paise (Rs 1.00 = 100 paise) held in long long. No floating point is used for money.
Concession	The brief gives students and senior citizens a concession but states no percentage or rule. No number is invented here; a DiscountPolicy extension point is designed and the missing rule is listed as a limitation.
Weekend (Task 12)	10% discount on the base fare. It **replaces** the peak surcharge and does not combine with it.

1.2 Goals

- Calculate every fare exactly and consistently.
- Reject an invalid hour, zone or recharge amount before any card is changed.
- Keep every business rule in one place so a tariff revision is a one-line change.
- Demonstrate correctness with boundary and invalid-input tests that really run.
- Compare a procedural design with an object-oriented one, then adapt the design to new requirements.

2. Task 1 – Understand the Problem and Identify the Objects

2.1 Objects, data and behaviour

Class	State (data)	Behaviour	Why it is a class
SmartCard	cardId, balancePaise	canTravel(), deduct(), recharge(), balance()	Owns the balance and is the single enforcer of the minimum-balance rule.
Passenger	passengerId, category (Regular / Student / Senior)	category()	Carries who is travelling, so concessions can be added later without touching the card.
Journey	zone, hour, weekend flag	validate()	Groups the inputs of one trip so they are validated together.
Zone (enum class)	Inner, Middle, Outer, Airport	—	Prevents free text or an arbitrary int being used as a zone.
FarePolicy	base-fare table, surcharge %, discount %, peak windows	isPeak(), fareFor()	The one place where all fare rules and constants live.
MonthlyPass	two covered zones, valid-from, valid-until	covers(zone, day)	Unlimited-travel entitlement is a different idea from stored value, so it is separate.

2.2 Procedural versus object-oriented design

In a procedural program the balance, zone, hour and fare are separate variables, and free functions such as calculateFare() and canTravel() operate on them. This is short and perfectly adequate for a classroom exercise, but nothing stops any function that can see the balance from subtracting a fare without checking the minimum, or from writing a negative number into it. As the program grows, the rule "balance must stay ≥ Rs 50" has to be remembered by every programmer in every function.

In the object-oriented design the balance is a **private** member of SmartCard. The only way to change it is through deduct() and recharge(), and both refuse invalid requests. The rule is written once and cannot be bypassed from outside the class. Fare rules similarly sit inside FarePolicy.

2.3 The four OOP principles in this system

- **Encapsulation** – balancePaise_ is private; deductions go through a validated method.
- **Abstraction** – callers use FarePolicy::fareFor(zone, hour, weekend) and SmartCard::deduct(fare) without knowing the arithmetic behind them.
- **Inheritance** – a Passenger base class could have StudentPassenger and SeniorPassenger subclasses. It should only be used if their behaviour really differs; if only a percentage differs, a policy object is simpler than a class hierarchy.

- **Polymorphism** – a DiscountPolicy interface with StandardDiscount, StudentDiscount and SeniorDiscount implementations lets FarePolicy apply a concession through one virtual call, without an if chain on passenger type.

2.4 Journey invariants

- The fare is never negative and is only computed for a valid zone and an hour from 0 to 23.
- The balance is stored in paise and can never become negative.
- A journey is approved only if balance − fare ≥ 5,000 paise.
- A denied journey leaves the balance unchanged.
- Peak windows are start-inclusive and end-exclusive.
- Every rate and threshold is defined exactly once as a named constant.

2.5 UML class sketch

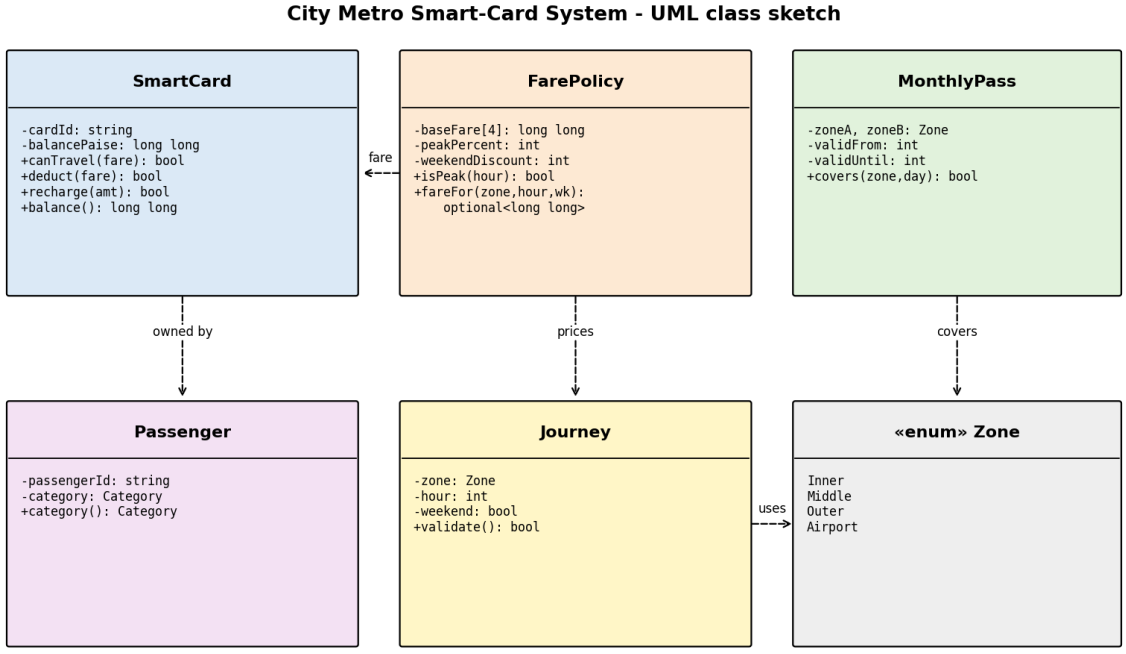


Figure 1: UML-style class sketch of the smart-card system

2.6 Complexity and alternative

For one journey both designs do a fixed number of comparisons and additions, so time is **O(1)**, and one card stores a fixed number of fields, so space is **O(1)** per card. The real difference is not speed but maintainability: in procedural code a tariff change may mean hunting through several functions and global variables, whereas in the OO design it means editing the FarePolicy constants.

3. Task 2 – Tokens, Keywords, Identifiers and Constants

3.1 Five token categories

Category	Examples from this project	Explanation
Keyword	class, enum, constexpr, return, switch, if	Reserved words with fixed meaning in C++.
Identifier	SmartCard, balancePaise_, fareFor, MIN_BALANCE_PAISE	Names chosen by the programmer. They must start with a letter or underscore and cannot be keywords.
Constant (literal)	20, 5000, 1000LL, true, 'A', "Rs "	Fixed values written directly in the source.
Operator	+ - * / % >= && || ?: -=	Perform arithmetic, comparison, logical or assignment actions.
Punctuator	{ } ( ) ; , :: < >	Structure declarations and statements.

3.2 Named constants used in the engine

```
enum class Zone { Inner, Middle, Outer, Airport };
constexpr long long MIN_BALANCE_PAISE       = 5000;
constexpr int       PEAK_SURCHARGE_PERCENT   = 20;
constexpr int       WEEKEND_DISCOUNT_PERCENT = 10;
constexpr std::array<long long, 4> BASE_FARE_PAISE = {1000, 2000, 3000, 6000};
```

constexpr guarantees the value is computed at compile time and is type-checked. const only says the object cannot be modified after initialisation (its value may still be computed at run time). #define is plain text substitution done by the preprocessor, so it has no type and no scope and can produce surprising results (for example #define FARE 10+5 makes FARE*2 equal 20, not 30). enum class is preferred over a plain enum because its names are scoped (Zone::Inner) and it does not silently convert to int.

Naming convention used: PascalCase for types (SmartCard), camelCase for functions and variables (calculateFare), trailing underscore for private members (balancePaise_), and UPPER_SNAKE_CASE for constants (MIN_BALANCE_PAISE).

3.3 Macro versus constexpr

Replacing the constants with #define macros would not change run-time cost: a fare lookup is still **O(1)** time and the values need no meaningful storage in either case. But macros ignore scope and types and give poor compiler error messages, so scoped enums with constexpr constants remain the better choice.

4. Task 3 – Choose Data Types and Check Type Compatibility

Value	Chosen type	Reason
Fare, balance, recharge amount	long long (paise)	Exact integer arithmetic with a very large range.
Zone	enum class Zone	Only named zones are valid.
Hour	int	Small whole number, validated to 0–23.
Peak / weekend flags	bool	Two-state conditions.
Journey count, daily totals	long long	Aggregates can exceed 2 billion paise.
Passenger category	enum class Category	Closed set of named values; safer than a raw char.
Loop index over a collection	std::size_t	Matches container sizes and avoids signed/unsigned warnings.
Analytical ratio (non-money)	double	Fine for statistics, never for stored money.

Money is held in integer paise: Rs 10.20 is stored as 1020. Many decimal fractions (for example 0.10) have no exact binary floating-point representation, so repeated additions of double values drift. Integer addition and subtraction are exact as long as the type does not overflow.

4.1 Measured type sizes (this machine)

Program: sizes.cpp

```
#include <climits>
#include <iostream>
int main() {
    std::cout << "sizeof(bool)      = " << sizeof(bool) << " byte(s)\n";
    std::cout << "sizeof(char)      = " << sizeof(char) << " byte(s)\n";
    std::cout << "sizeof(int)       = " << sizeof(int) << " byte(s)\n";
    std::cout << "sizeof(long long) = " << sizeof(long long) << " byte(s)\n";
    std::cout << "sizeof(float)     = " << sizeof(float) << " byte(s)\n";
    std::cout << "sizeof(double)    = " << sizeof(double) << " byte(s)\n";
    std::cout << "INT_MAX           = " << INT_MAX << '\n';
    std::cout << "LLONG_MAX         = " << LLONG_MAX << '\n';
    long long worst = 3000000LL * 3600LL;   // 3,000,000 trips x Rs 36 in paise
    std::cout << "Worst daily total = " << worst << " paise ("
              << (worst > INT_MAX ? "exceeds" : "fits in") << " INT_MAX)\n";
}
```

Output (actual run)

```
sizeof(bool)      = 1 byte(s)
sizeof(char)      = 1 byte(s)
sizeof(int)       = 4 byte(s)
sizeof(long long) = 8 byte(s)
sizeof(float)     = 4 byte(s)
sizeof(double)    = 8 byte(s)
INT_MAX           = 2147483647
LLONG_MAX         = 9223372036854775807
Worst daily total = 10800000000 paise (exceeds INT_MAX)
```

At network scale the worst case is 3,000,000 journeys × Rs 36 = Rs 108,000,000 = **10,800,000,000 paise**. That is larger than INT_MAX (2,147,483,647) so a 32-bit int counting paise would overflow, whereas the same amount in long long fits comfortably. The output above confirms the comparison on this machine. Signed integer overflow is undefined behaviour in C++, so the program must prevent it by choosing a wide enough type, not rely on wrap-around.

4.2 Double alternative

Using double rupees keeps time at O(1) per addition and uses 8 bytes per value, the same as long long, but each addition can introduce a tiny representation error. The size of the error depends on the values and their order, so it is measured, not assumed — see Task 10 for the experiment.

5. Task 4 – Control Implicit Conversions

Implicit conversions happen when an integer expression is assigned to a double, when char or short values are promoted to int in arithmetic, when a non-zero integer is converted to bool, and when signed and unsigned integers are compared. The last one is dangerous because the negative signed value is converted to a very large unsigned value.

5.1 Predict-the-output program (actual result)

Program: conv.cpp

```
#include <iostream>
int main() {
    int a = 9, b = 4;
    double c = a / b;                       // integer division first
    double d = static_cast<double>(a) / b;    // real division
    std::cout << c << ' ' << d << '\n';
    char ch = 'B' + 3;
    std::cout << ch << ' ' << ch + 1 << '\n';
    bool flag = -7;
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';
    unsigned int u = 2;
    int s = -2;
    std::cout << (s < u ? "s < u" : "s >= u") << '\n';
    int safeCount{3};
    std::cout << "safeCount = " << safeCount << '\n';
    double price = 19.99;
    int truncated = static_cast<int>(price);
    std::cout << "static_cast<int>(19.99) = " << truncated << '\n';
}
```

Prediction before running: 9/4 is integer division, so c is 2 (printed as 2) and d is 2.25. 'B' + 3 is 'E', and ch + 1 is promoted to int giving 70. -7 is non-zero so flag is true (1, then true with boolalpha). For s < u, -2 becomes a huge unsigned number, so the comparison is false and the program prints s >= u. Finally static_cast<int>(19.99) truncates to 19.

Output (actual run)

```
2 2.25
E 70
1 true
s >= u
safeCount = 3
static_cast<int>(19.99) = 19
```

The output matches every prediction. The compiler also warned about the risky comparison when compiled with -Wall -Wextra:

Compiler warning (real)

```
conv.cpp:13:21: warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare]
   13 |     std::cout << (s < u ? "s < u" : "s >= u") << '\n';
```

The warning -Wsign-compare is exactly the signed/unsigned problem described above. The fix is to compare values of the same signedness, or to cast deliberately with static_cast.

5.2 Safer conversions

```
double ratio = static_cast<double>(a) / b;   // intent is visible
int safeCount{3};                            // valid
// int bad{3.7};                             // compile error: narrowing
bool enabled = (inputValue != 0);            // explicit comparison
```

Brace initialisation rejects narrowing at compile time, and explicit casts document intent. They should be used where a conversion is intentional, not sprinkled on every expression. For valid inputs they do not change time or memory complexity.

6. Task 5 – Operators and Precedence

6.1 Fare expression

```
bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
long long surcharge = peak ? basePaise * 20 / 100 : 0;
long long finalFare = basePaise + surcharge;
```

&& has higher precedence than ||, so the brackets around each window are not strictly required, but they make the two windows obvious to a reviewer and avoid the compiler warning -Wparentheses. The conditional operator ?: selects one of two values. basePaise * 20 / 100 multiplies first and divides second, so integer truncation happens only once, at the end.

6.2 Increment and remainder

- x++ yields the old value then increments; ++x increments first then yields the new value.
- Avoid writing i = i++; it is confusing and was undefined behaviour before C++17. Use a separate increment statement.
- Integer division truncates toward zero, and % satisfies a == (a / b) * b + a % b. So -17 / 5 is -3 and -17 % 5 is -2.

6.3 Predict-the-output program (actual result)

Program: ops.cpp

```
#include <iostream>
int main() {
    int x = 6, y = 4;
    std::cout << x + y * 3 << ' ' << (x + y) * 3 << '\n';
    int z = x++ + ++y;
    std::cout << z << ' ' << x << ' ' << y << '\n';
    std::cout << (x > y ? x : y) << ' ' << x % 5 * 3 << '\n';
    std::cout << 17 / 5 << ' ' << 17 % 5 << ' ' << -17 / 5 << ' ' << -17 % 5 << '\n';
    int k = 0;
    for (int i = 1; i <= 10; i += 3) k += i;
    std::cout << k << '\n';
}
```

Output (actual run)

```
18 30
11 7 5
7 6
3 2 -3 -2
22
```

Line-by-line reasoning. Line 1: 6 + 4*3 = 18 and (6+4)*3 = 30. Line 2: x++ contributes 6 then x becomes 7; ++y makes y 5 and contributes 5, so z = 11, x = 7, y = 5. Line 3: x > y is true so the result is 7; x % 5 * 3 is evaluated left to right as (7 % 5) * 3 = 6. Line 4: 17/5 = 3, 17%5 = 2, -17/5 = -3, -17%5 = -2. Line 5: i takes 1, 4, 7, 10 so k = 22.

6.4 Lookup-table alternative

A `bool peak[24]` table gives an O(1) lookup using 24 bytes (since `sizeof(bool)` is 1 on this machine). The comparison expression is also O(1) and needs no storage. A table is better when peak hours are loaded from configuration or the schedule is complex; for two fixed windows the expression is simpler and cannot get out of step with a data file.

7. Task 6 – Design the Control Structures

7.1 Pseudocode

```
ALGORITHM CalculateFare(zone, hour, weekend)
  IF hour < 0 OR hour > 23 THEN RETURN INVALID
  IF zone is not a known zone THEN RETURN INVALID
  base <- BASE_FARE[zone]
  IF weekend THEN
     RETURN base - (base * 10 DIV 100)
  ELSE IF (8 <= hour < 10) OR (17 <= hour < 20) THEN
     RETURN base + (base * 20 DIV 100)
  ELSE
     RETURN base
  ENDIF
END ALGORITHM

ALGORITHM ProcessJourney(card, journey)
  fare <- CalculateFare(journey.zone, journey.hour, journey.weekend)
  IF fare is INVALID THEN REPORT "invalid journey"; RETURN
  IF card.balance - fare >= 5000 THEN
     card.balance <- card.balance - fare
     REPORT approved and new balance
  ELSE
     REPORT denied; balance unchanged
  ENDIF
END ALGORITHM
```

Choice of structures. switch suits a small discrete enumeration such as zone; if / else if suits ranges (the hour) and validation; ?: suits a short two-way value choice. A for loop processes a known number of journeys, a range-based for walks a collection of trips, and a while loop reads input until end-of-file or a sentinel. continue skips a rejected record and break leaves a loop when a stop condition occurs.

7.2 Implementation

The fare rule is implemented in FarePolicy::fareFor, which returns std::optional<long long>: a value for a valid request and std::nullopt for an invalid one, so an invalid zone or hour can never be mistaken for a fare of zero.

Extract from metro.cpp (full listing in Appendix A)

```
class FarePolicy {
public:
   static bool isPeak(int hour) {
      return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
   }
   static std::optional<long long> fareFor(Zone zone, int hour, bool weekend,
                                           bool airportEnabled = true) {
      if (hour < 0 || hour > 23) return std::nullopt;
      auto idx = static_cast<std::size_t>(zone);
      if (idx >= BASE_FARE_PAISE.size()) return std::nullopt;
      if (zone == Zone::Airport && !airportEnabled) return std::nullopt;
      long long base = BASE_FARE_PAISE[idx];
      if (weekend)      return base - base * WEEKEND_DISCOUNT_PERCENT / 100;
      if (isPeak(hour)) return base + base * PEAK_SURCHARGE_PERCENT / 100;
      return base;
   }
```

```
};
```

7.3 Switch versus array

Both a `switch` and an array lookup are O(1) for a fixed number of zones. A `switch` makes each case explicit and the compiler can warn if an enum value is missing. An array is shorter and easier to extend but depends on the enum order matching the array order. Adding a fourth zone means changing the enum, the fare table, the validation, the tests and the documentation, which is why the table is kept in one place.

8. Task 7 – From Procedural to Object-Oriented

8.1 Procedural version

```
bool canTravelProcedural(long long balance, long long fare) {
    return fare >= 0 && balance - fare >= 5000;
}
bool deductProcedural(long long& balance, long long fare) {
    if (!canTravelProcedural(balance, fare)) return false;
    balance -= fare;
    return true;
}
```

8.2 Object-oriented version

```
class SmartCard {
    std::string id_;
    long long balancePaise_;                    // private: cannot be set from outside
public:
    SmartCard(std::string id, long long initial)
        : id_(std::move(id)), balancePaise_(initial >= 0 ? initial : 0) {}
    long long balance() const { return balancePaise_; }
    bool canTravel(long long fare) const {
        return fare >= 0 && balancePaise_ - fare >= MIN_BALANCE_PAISE;
    }
    bool deduct(long long fare) {
        if (!canTravel(fare)) return false;
        balancePaise_ -= fare;
        return true;
    }
    bool recharge(long long amount) {
        if (amount <= 0) return false;
        balancePaise_ += amount;
        return true;
    }
};
```

Both versions take O(1) time and O(1) space per check. The difference is protection of the invariant: with the procedural version any code holding the balance variable can set it to -100; with the class it cannot compile. The constructor also clamps a negative opening balance to zero (tested in Task 9).

9. Task 8 – Read and Print Data Safely

9.1 Input validation loop

Program: input.cpp

```
#include <iostream>
#include <limits>
int main() {
    int hour;
    while (true) {
        std::cout << "Enter hour (0-23): ";
        if (std::cin >> hour) {
            if (hour >= 0 && hour <= 23) break;
            std::cout << "Hour must be from 0 to 23.\n";
        } else {
            if (std::cin.eof()) return 1;
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "Invalid input. Enter a number.\n";
        }
    }
    std::cout << "Accepted hour = " << hour << '\n';
}
```

When `cin >> hour` meets letters, the stream enters a fail state and the bad characters stay in the buffer. `clear()` resets the state and `ignore()` throws away the rest of the line, otherwise the same bad text would be read forever. Tested with the input lines `abc`, `25`, `-3`, `9`:

Output (actual run)

```
Enter hour (0-23): Invalid input. Enter a number.
Enter hour (0-23): Hour must be from 0 to 23.
Enter hour (0-23): Hour must be from 0 to 23.
Enter hour (0-23): Accepted hour = 9
```

The program rejects text, rejects out-of-range numbers on both sides (25 and -3), and accepts 9. (Note: the prompt text and the answers appear on the same line because the redirected input is not echoed.)

9.2 Money formatting

```
std::string rupees(long long paise) {
    std::string s = "Rs " + std::to_string(paise / 100) + ".";
    long long p = paise % 100;
    if (p < 10) s += "0";           // pad so 5 paise prints as .05
    return s + std::to_string(p);
}
```

Formatting is done from integers (rupees = paise / 100, remainder = paise % 100), so no floating-point rounding can alter what is displayed. For whole-line input, `getline()` followed by `stoi()` in a `try/catch` can additionally reject trailing junk such as `12abc`. Both approaches take O(L) time for a line of L characters; `getline` needs O(L) buffer space.

10. Task 9 – Implement and Test the Fare Logic

10.1 Test strategy

The tests cover normal fares, both ends of each peak window, the last peak minute (19:59), weekend rules, the Airport zone, invalid hour and zone, the exact minimum-balance threshold, a negative recharge and a negative opening balance. The fare tests are table-driven: each row holds the name, inputs and the expected fare, and the loop prints PASS or FAIL.

10.2 Actual test run

Output of metro.cpp, Parts 1–3

```
=== PART 1: Table-driven fare tests ===
T1  Inner  07:00 weekday     expected Rs 10.00  actual Rs 10.00  PASS
T2  Inner  08:00 weekday     expected Rs 12.00  actual Rs 12.00  PASS
T3  Middle 09:00 weekday     expected Rs 24.00  actual Rs 24.00  PASS
T4  Middle 10:00 weekday     expected Rs 20.00  actual Rs 20.00  PASS
T5  Outer  17:00 weekday     expected Rs 36.00  actual Rs 36.00  PASS
T6  Outer  19:59 weekday     expected Rs 36.00  actual Rs 36.00  PASS
T7  Outer  20:00 weekday     expected Rs 30.00  actual Rs 30.00  PASS
T8  Airport 08:00 weekday    expected Rs 72.00  actual Rs 72.00  PASS
T9  Airport 08:00 weekend    expected Rs 54.00  actual Rs 54.00  PASS
T10 Inner  12:00 weekend     expected Rs 9.00   actual Rs 9.00   PASS
10/10 fare tests passed

=== PART 2: Invalid input tests ===
hour 24      -> rejected
hour -1      -> rejected
zone value 9 -> rejected

=== PART 3: SmartCard boundary tests ===
Rs 60.00, Rs 10.00 fare -> approved, balance Rs 50.00
Rs 59.99, Rs 10.00 fare -> denied, balance Rs 59.99
recharge(-100)  -> rejected, balance Rs 100.00
recharge(2500)  -> accepted, balance Rs 125.00
constructor with -500 paise -> balance Rs 0.00
```

All 10 fare tests passed. The boundary tests show that Rs 60.00 with a Rs 10.00 fare is approved and leaves exactly Rs 50.00, whereas Rs 59.99 is denied and unchanged. Invalid inputs were all rejected and the negative recharge did not alter the balance.

10.3 Five-journey simulation

Starting balance Rs 150.00. Trip 3 leaves Rs 60.00, so trip 4 (Rs 12.00 would leave Rs 48.00) is denied, but trip 5 (Rs 10.00 leaves exactly Rs 50.00) is approved. This demonstrates both sides of the inclusive threshold in one run.

Output of metro.cpp, Part 4

```
=== PART 4: Five-journey simulation (start Rs 150.00) ===
Trip 1: Middle at 09:00  fare Rs 24.00  -> APPROVED, balance Rs 126.00
Trip 2: Outer  at 14:00  fare Rs 30.00  -> APPROVED, balance Rs 96.00
Trip 3: Outer  at 18:00  fare Rs 36.00  -> APPROVED, balance Rs 60.00
Trip 4: Inner  at 08:00  fare Rs 12.00  -> DENIED,   balance Rs 60.00
Trip 5: Inner  at 21:00  fare Rs 10.00  -> APPROVED, balance Rs 50.00
Total collected: Rs 100.00, final balance: Rs 50.00
```

Trip	Zone	Hour	Fare	Result	Balance
Start	—	—	—	—	Rs 150.00
1	Middle	09:00 (peak)	Rs 24.00	Approved	Rs 126.00
2	Outer	14:00	Rs 30.00	Approved	Rs 96.00
3	Outer	18:00 (peak)	Rs 36.00	Approved	Rs 60.00
4	Inner	08:00 (peak)	Rs 12.00	Denied (would leave Rs 48.00)	Rs 60.00
5	Inner	21:00	Rs 10.00	Approved (leaves exactly Rs 50.00)	Rs 50.00

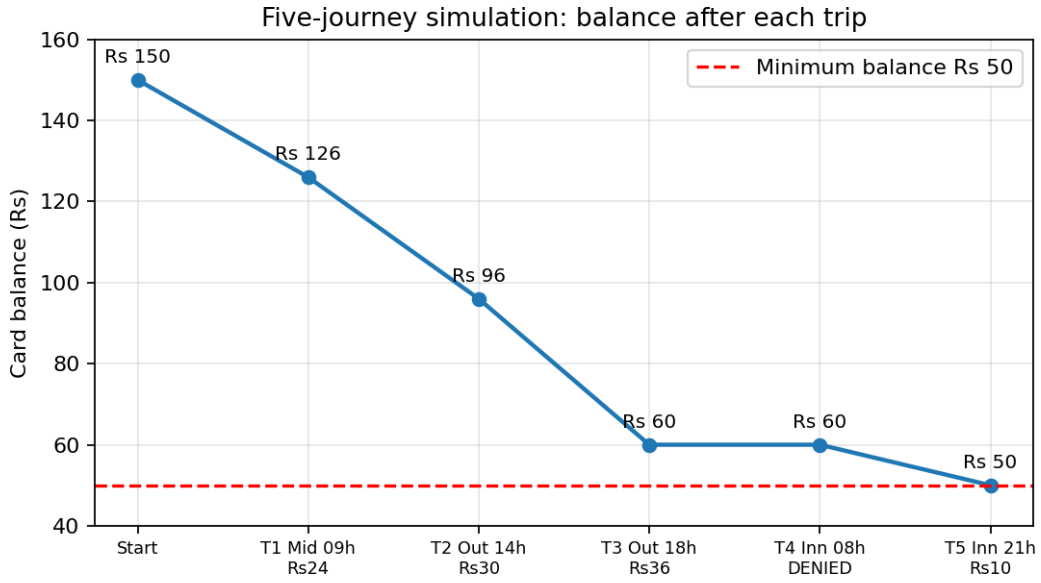


Figure 2: Card balance after each journey of the simulation

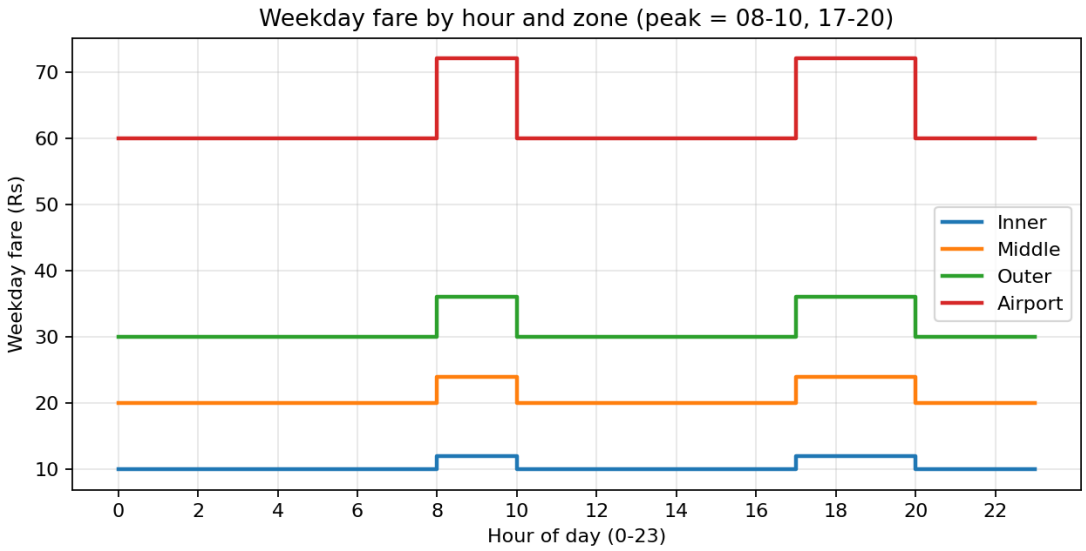


Figure 3: Weekday fare by hour for each zone (peaks at 08:00–10:00 and 17:00–20:00)

10.4 Complexity of the test approach

A table-driven test loop takes O(t) time and O(t) space for t cases, and adding a new boundary case is a one-line change rather than a new block of code.

11. Task 10 – Compare Expectations with Execution

11.1 Scale analysis

Journeys/day	All at Rs 10	All at Rs 20	All at Rs 30	All Rs 30 trips at peak (Rs 36)
20,000	Rs 200,000	Rs 400,000	Rs 600,000	Rs 720,000
500,000	Rs 5,000,000	Rs 10,000,000	Rs 15,000,000	Rs 18,000,000
3,000,000	Rs 30,000,000	Rs 60,000,000	Rs 90,000,000	Rs 108,000,000

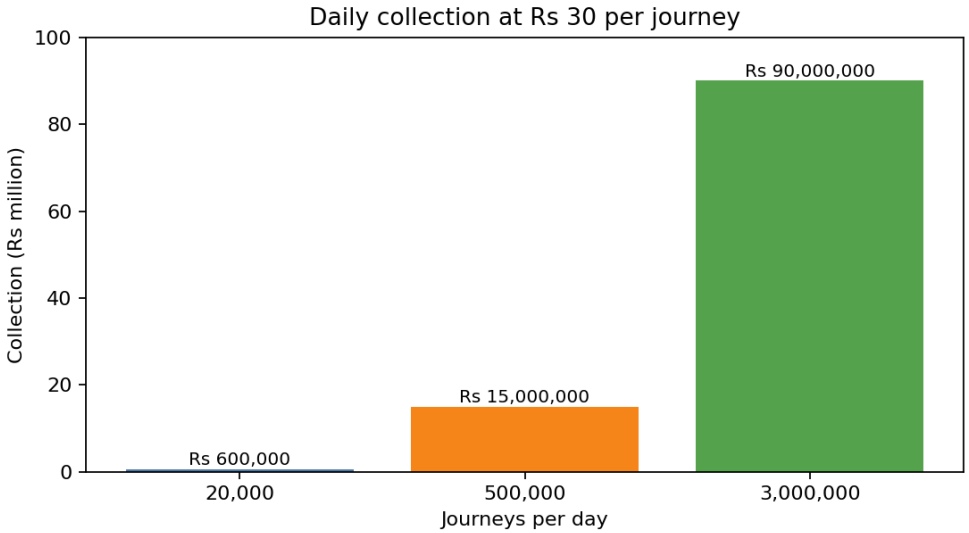


Figure 4: Daily collection at Rs 30 per journey for the three scales

11.2 Measured experiment: integer paise versus floating point

Program: acc.cpp (Part A shown; full listing in Appendix A)

```
#include <climits>
#include <iomanip>
#include <iostream>
#include <vector>
int main() {
    // Part A: 0.10 added 10,000,000 times (Rs 0.10 = 10 paise)
    long long p = 0; double d = 0.0; float fl = 0.0f;
    for (int i = 0; i < 10000000; ++i) { p += 10; d += 0.10; fl += 0.10f; }
    std::cout << std::fixed << std::setprecision(6);
    std::cout << "Integer paise total : " << p << " paise = Rs " << p / 100.0 << "\n";
    std::cout << "double total        : Rs " << d << "    error = " << (d - 1000000.0) << "\n";
    std::cout << "float total         : Rs " << fl << "    error = " << (fl - 1000000.0f) << "\n";
```

Output of acc.cpp (actual run)

```
Integer paise total : 100000000 paise = Rs 1000000.000000
double total        : Rs 999999.999839    error = -0.000161
float total         : Rs 1087937.000000    error = 87937.000000
3,000,000 journeys, fares cycle Rs 10/12/20/24/30/36
long long paise : 6600000000 (Rs 66000000.00)
double rupees   : 66000000.000000
paise total > INT_MAX ? true  (INT_MAX = 2147483647)
```

```
Daily collection at Rs 30 per journey:
  20000 journeys -> Rs 600000
 500000 journeys -> Rs 15000000
3000000 journeys -> Rs 90000000
```

Observations. (1) Adding 10 paise ten million times gives exactly 100,000,000 paise in long long. (2) The same sum in double (0.10 per step) ends at 999,999.999839, an error of −0.000161 rupees: tiny, but not zero, and it would make a reconciliation against the ledger fail. (3) float is far worse: it stopped growing correctly and produced 1,087,937 instead of 1,000,000, an error of 87,937 rupees. This happens because once the running total is large, adding 0.10 is smaller than the spacing between representable float values. (4) The day total for 3,000,000 mixed-fare journeys is Rs 66,000,000, and the paise total 6,600,000,000 exceeds INT_MAX, confirming that long long is required.

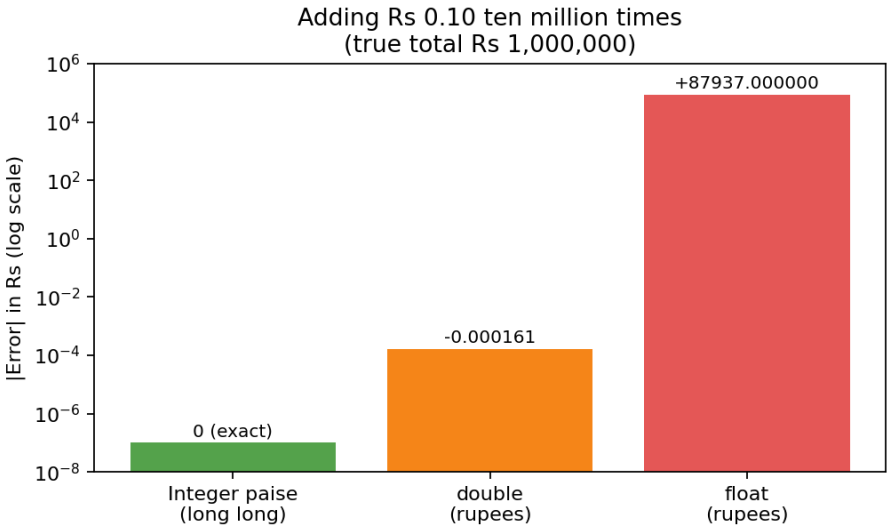


Figure 5: Absolute error after adding Rs 0.10 ten million times (log scale)

Complexity. The accumulation loop is O(n) time for n journeys and O(1) space whichever type is used. Integer and double additions are each O(1). The type changes accuracy and overflow safety, not asymptotic cost. The compile with -Wall -Wextra produced no warnings for acc.cpp or metro.cpp, and the only warning among all programs was the intentional -Wsign-compare in conv.cpp.

11.3 A bug found while testing

While writing the simulation, the first run printed the time as 90:00 instead of 09:00. The cause was that std::left had been set earlier for the test table, so setw(2) with fill 0 padded on the right. The fix was to switch to std::right for the time field and restore std::left afterwards. This is a good example of why output formatting must be tested and not only the arithmetic.

12. Task 11 – Recommend and Defend the Final Design

12.1 Recommended design

- Store fares, balances and totals as long long paise.
- Use enum class Zone and constexpr constants for the fare rules.
- Use SmartCard with a private balance and validated deduct() / recharge().
- Put peak-window and fare logic only in FarePolicy.
- Validate every input: reject invalid hours, zones and recharge amounts before anything changes.
- Use table-driven tests and keep the real compiler output with the report.
- Process journeys in a stream: each is O(1), so n journeys take O(n) time.

12.2 Revision procedure

To revise a tariff, change the constants (or the configuration table) in one place and add or update the matching test rows. Nothing else in the program contains a fare number, so no hunting is needed. The design suits millions of journeys: each is constant time, the day is O(n), and if journeys are streamed the working memory stays O(1) apart from logs.

12.3 Limitations

- The brief does not state the student/senior concession percentage, how it rounds, or whether it combines with the peak surcharge. These must be clarified before the concession is implemented; the DiscountPolicy interface is the planned extension point.
- Refunds, concurrency, database failure, card security, payment settlement and audit logging are not modelled.
- The pass model uses simple day numbers; a production system needs a real date type and rules for how a pass and stored value interact.

12.4 Configuration-file alternative

Loading z fares from a file costs O(z) at start-up and O(z) storage; each lookup stays O(1). It avoids recompiling for fare-only changes but needs validation, versioning, safe defaults and failure handling. constexpr is simpler for a teaching program; a validated configuration service is more suitable for a real operator with frequent tariff changes.

13. Task 12 – Day-4 Design Challenge

New requirements: (1) an **Airport** zone with a base fare of Rs 60; (2) a **10% weekend discount** that does not combine with the peak surcharge; (3) a **monthly pass** giving unlimited travel in two chosen zones.

13.1 A. Impact analysis

Component	Change required	Reason
Zone enum	Add Airport	New valid zone.
Fare table	Add base fare 6000 paise (and enlarge array to 4)	Rs 60.00 base fare.
FarePolicy	Add weekend discount (10%) with "replaces peak" rule	Discount must be decided in one place.
Journey	Add weekend flag (or a date)	The policy must know the day type.
SmartCard	No change to balance logic; entitlement queried separately	Stored value and pass are different concepts.
MonthlyPass (new)	Two covered zones plus validity period	Unlimited travel only in two zones during validity.
Tests	Airport, weekend, weekend+peak, pass-covered and pass-not-covered cases	New boundaries and interactions.

13.2 B. Evaluate the original design

The original structure survives well: SmartCard encapsulation and integer paise are untouched, and only the policy and a new entitlement class change. A switch could still handle four zones, but an array indexed by a validated enum scales better. The pass is kept out of SmartCard because it has its own dates and zones and should not be a boolean flag on the card. The weekend rule is written as an explicit policy decision: if the day is a weekend, apply the discount and skip the surcharge.

Worked examples (discount applied to base fare; paise arithmetic truncates): Airport weekday peak = Rs 60 + 20% = **Rs 72.00**; Airport weekend peak hour = Rs 60 − 10% = **Rs 54.00** (no surcharge); Inner weekend = Rs 10 − 10% = **Rs 9.00**.

13.3 C. Revised design and code

```
enum class Zone { Inner, Middle, Outer, Airport };
constexpr std::array<long long, 4> BASE_FARE_PAISE = {1000, 2000, 3000, 6000};
constexpr int PEAK_SURCHARGE_PERCENT   = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;


// In FarePolicy::fareFor:  weekend discount replaces the peak surcharge
if (weekend)      return base - base * WEEKEND_DISCOUNT_PERCENT / 100;
if (isPeak(hour)) return base + base * PEAK_SURCHARGE_PERCENT / 100;
return base;


class MonthlyPass {
```

```
    Zone a_, b_;  int from_, until_;
public:
    MonthlyPass(Zone a, Zone b, int from, int until)
        : a_(a), b_(b), from_(from), until_(until) {}
    bool covers(Zone z, int day) const {
        return day >= from_ && day <= until_ && (z == a_ || z == b_);
    }
};
```

These classes are part of the same metro.cpp that was compiled and run for Task 9, so the revised design is tested, not only described. A production version should use a real date type, prevent duplicate covered zones, and define how a valid pass interacts with the stored balance.

13.4 D. Evidence-based defence

Revised-design fare tests (from the real run)

```
T8  Airport 08:00 weekday    expected Rs 72.00  actual Rs 72.00  PASS
T9  Airport 08:00 weekend    expected Rs 54.00  actual Rs 54.00  PASS
T10 Inner  12:00 weekend     expected Rs 9.00   actual Rs 9.00   PASS
```

MonthlyPass tests (from the real run)

```
=== PART 5: MonthlyPass tests (Inner+Middle, days 1-30) ===
Inner,  day 15 covered: true
Outer,  day 15 covered: false
Middle, day 31 covered: false
```

Test	Expected	Design point validated
Inner, weekday 08:00	Rs 12.00	Peak starts at the inclusive boundary.
Outer, weekday 20:00	Rs 30.00	Peak ends at the exclusive boundary.
Airport, weekday 08:00	Rs 72.00	New zone combines correctly with peak.
Airport, weekend 08:00	Rs 54.00	Weekend discount replaces the surcharge.
Inner, weekend 12:00	Rs 9.00	10% discount applied.
Pass: Inner day 15	Covered	Zone and date both valid.
Pass: Outer day 15	Not covered	Zone outside the pass.
Pass: Middle day 31	Not covered	Date outside validity.

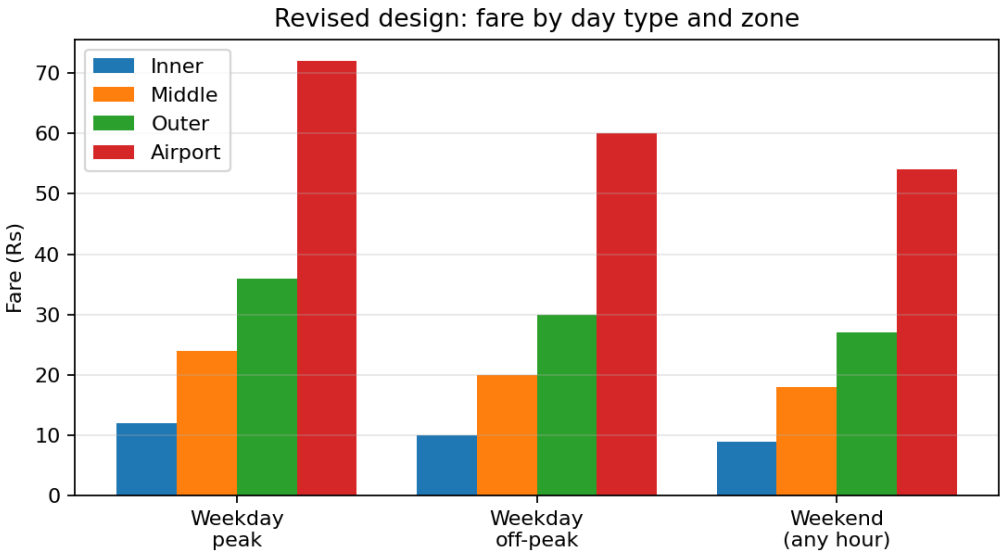


Figure 6: Fare by day type for all four zones in the revised design

Decision: MODIFY, do not redesign. The encapsulated SmartCard, the integer-paise approach and the central fare table all remain valid. Only the enum, one table entry, one weekend rule and one new class were needed, and the existing fare tests (T1–T7) still pass unchanged, which is the strongest evidence that the original design was sound.

14. Overall Complexity, Limitations and Conclusion

Operation	Time	Space
Single fare calculation	O(1)	O(1)
Single balance check	O(1)	O(1)
Process n journeys sequentially	O(n)	O(1) working (excluding logs)
Run t table-driven tests	O(t)	O(t)
Load z fares from a configuration file	O(z) start-up, O(1) lookup	O(z)
MonthlyPass coverage check	O(1)	O(1)

The recommended design stores money as exact integer paise, keeps every rate in a named constant, uses a scoped enum for zones, states peak boundaries explicitly, and hides the balance inside a class that cannot be bypassed. Real compiled runs confirmed all fare, boundary, invalid-input and pass tests, and the measurements showed concretely why `double` and especially `float` are unsuitable for money and why `int` is too small for paise totals at network scale. The extended design absorbed the Airport zone, the weekend discount and monthly passes with changes confined to the fare policy and one new class.

Appendix A – Source code

metro.cpp (compile: g++ -std=c++17 -Wall -Wextra -o metro metro.cpp)

```
// City Metro Smart-Card Fare Engine  (C++17)
#include <array>
#include <cstddef>
#include <iomanip>
#include <iostream>
#include <optional>
#include <string>
#include <vector>


enum class Zone { Inner, Middle, Outer, Airport };


constexpr long long MIN_BALANCE_PAISE      = 5000;   // Rs 50.00
constexpr int       PEAK_SURCHARGE_PERCENT = 20;
constexpr int       WEEKEND_DISCOUNT_PERCENT = 10;
constexpr std::array<long long, 4> BASE_FARE_PAISE = {1000, 2000, 3000, 6000};


std::string zoneName(Zone z) {
    switch (z) {
        case Zone::Inner:   return "Inner";
        case Zone::Middle:  return "Middle";
        case Zone::Outer:   return "Outer";
        case Zone::Airport: return "Airport";
    }
    return "Unknown";
}


std::string rupees(long long paise) {
    std::string s = "Rs " + std::to_string(paise / 100) + ".";
    long long p = paise % 100;
    if (p < 10) s += "0";
    return s + std::to_string(p);
}


class FarePolicy {
public:
    static bool isPeak(int hour) {
        return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
    }
    // Returns nullopt for invalid input. Weekend discount REPLACES peak surcharge.
    static std::optional<long long> fareFor(Zone zone, int hour, bool weekend,
                                            bool airportEnabled = true) {
        if (hour < 0 || hour > 23) return std::nullopt;
        auto idx = static_cast<std::size_t>(zone);
        if (idx >= BASE_FARE_PAISE.size()) return std::nullopt;
        if (zone == Zone::Airport && !airportEnabled) return std::nullopt;
        long long base = BASE_FARE_PAISE[idx];
        if (weekend)        return base - base * WEEKEND_DISCOUNT_PERCENT / 100;
        if (isPeak(hour))   return base + base * PEAK_SURCHARGE_PERCENT / 100;
        return base;
    }
};


class SmartCard {
    std::string id_;
    long long balancePaise_;
```

```
public:
    SmartCard(std::string id, long long initial)
        : id_(std::move(id)), balancePaise_(initial >= 0 ? initial : 0) {}
    long long balance() const { return balancePaise_; }
    const std::string& id() const { return id_; }
    bool canTravel(long long fare) const {
        return fare >= 0 && balancePaise_ - fare >= MIN_BALANCE_PAISE;
    }
    bool deduct(long long fare) {
        if (!canTravel(fare)) return false;
        balancePaise_ -= fare;
        return true;
    }
    bool recharge(long long amount) {
        if (amount <= 0) return false;
        balancePaise_ += amount;
        return true;
    }
};

class MonthlyPass {
    Zone a_, b_;
    int from_, until_;
public:
    MonthlyPass(Zone a, Zone b, int from, int until) : a_(a), b_(b), from_(from), until_(until) {}
    bool covers(Zone z, int day) const {
        return day >= from_ && day <= until_ && (z == a_ || z == b_);
    }
};

struct FareCase { std::string name; Zone zone; int hour; bool weekend; long long expected; };

int main() {
    std::cout << "=== PART 1: Table-driven fare tests ===\n";
    std::vector<FareCase> cases = {
        {"T1  Inner  07:00 weekday", Zone::Inner,   7,  false, 1000},
        {"T2  Inner  08:00 weekday", Zone::Inner,   8,  false, 1200},
        {"T3  Middle 09:00 weekday", Zone::Middle,  9,  false, 2400},
        {"T4  Middle 10:00 weekday", Zone::Middle,  10, false, 2000},
        {"T5  Outer  17:00 weekday", Zone::Outer,   17, false, 3600},
        {"T6  Outer  19:59 weekday", Zone::Outer,   19, false, 3600},
        {"T7  Outer  20:00 weekday", Zone::Outer,   20, false, 3000},
        {"T8  Airport 08:00 weekday", Zone::Airport, 8, false, 7200},
        {"T9  Airport 08:00 weekend", Zone::Airport, 8, true,  5400},
        {"T10 Inner  12:00 weekend", Zone::Inner,   12, true,  900},
    };
    int pass = 0;
    for (const auto& c : cases) {
        auto f = FarePolicy::fareFor(c.zone, c.hour, c.weekend);
        bool ok = f && *f == c.expected;
        pass += ok;
        std::cout << std::left << std::setw(28) << c.name << " expected " << std::setw(9)
                  << rupees(c.expected) << " actual " << std::setw(9)
                  << (f ? rupees(*f) : "INVALID") << (ok ? " PASS\n" : " FAIL\n");
    }
    std::cout << pass << "/" << cases.size() << " fare tests passed\n";

    std::cout << "\n=== PART 2: Invalid input tests ===\n";
    auto bad1 = FarePolicy::fareFor(Zone::Inner, 24, false);
    auto bad2 = FarePolicy::fareFor(Zone::Inner, -1, false);
```

```
    auto bad3 = FarePolicy::fareFor(static_cast<Zone>(9), 10, false);
    std::cout << "hour 24      -> " << (bad1 ? "accepted (BUG)" : "rejected") << '\n';
    std::cout << "hour -1      -> " << (bad2 ? "accepted (BUG)" : "rejected") << '\n';
    std::cout << "zone value 9 -> " << (bad3 ? "accepted (BUG)" : "rejected") << '\n';

    std::cout << "\n=== PART 3: SmartCard boundary tests ===\n";
    SmartCard c1("C-001", 6000);
    std::cout << "Rs 60.00, Rs 10.00 fare -> " << (c1.deduct(1000) ? "approved" : "denied")
              << ", balance " << rupees(c1.balance()) << '\n';
    SmartCard c2("C-002", 5999);
    std::cout << "Rs 59.99, Rs 10.00 fare -> " << (c2.deduct(1000) ? "approved" : "denied")
              << ", balance " << rupees(c2.balance()) << '\n';
    SmartCard c3("C-003", 10000);
    std::cout << "recharge(-100)  -> " << (c3.recharge(-100) ? "accepted" : "rejected")
              << ", balance " << rupees(c3.balance()) << '\n';
    std::cout << "recharge(2500)  -> " << (c3.recharge(2500) ? "accepted" : "rejected")
              << ", balance " << rupees(c3.balance()) << '\n';
    SmartCard c4(  "C-004", -500);
    std::cout << "constructor with -500 paise -> balance " << rupees(c4.balance()) << '\n';

    std::cout << "\n=== PART 4: Five-journey simulation (start Rs 150.00) ===\n";
    SmartCard card("C-100", 15000);
    struct Trip { Zone z; int h; };
    std::vector<Trip> trips = {{Zone::Middle, 9}, {Zone::Outer, 14}, {Zone::Outer, 18},
                               {Zone::Inner, 8}, {Zone::Inner, 21}};
    int n = 1;
    long long collected = 0;
    for (const auto& t : trips) {
        auto f = FarePolicy::fareFor(t.z, t.h, false);
        std::cout << "Trip " << n++ << ": " << std::setw(6) << zoneName(t.z) << " at "
                  << std::right << std::setfill('0') << std::setw(2) << t.h << ":00" << std::setfill(' ') << std::left
                  << "  fare " << rupees(*f) << "  -> ";
        if (card.deduct(*f)) { collected += *f; std::cout << "APPROVED, balance " << rupees(card.balance()) << '\n'; }
        else std::cout << "DENIED,   balance " << rupees(card.balance()) << '\n';
    }
    std::cout << "Total collected: " << rupees(collected) << ", final balance: " << rupees(card.balance()) << '\n';

    std::cout << "\n=== PART 5: MonthlyPass tests (Inner+Middle, days 1-30) ===\n";
    MonthlyPass pass1(Zone::Inner, Zone::Middle, 1, 30);
    std::cout << "Inner,  day 15 covered: " << std::boolalpha << pass1.covers(Zone::Inner, 15) << '\n';
    std::cout << "Outer,  day 15 covered: " << pass1.covers(Zone::Outer, 15) << '\n';
    std::cout << "Middle, day 31 covered: " << pass1.covers(Zone::Middle, 31) << '\n';
    return 0;
}
```

acc.cpp

```
#include <climits>
#include <iomanip>
#include <iostream>
#include <vector>
int main() {
    // Part A: 0.10 added 10,000,000 times (Rs 0.10 = 10 paise)
    long long p = 0; double d = 0.0; float fl = 0.0f;
    for (int i = 0; i < 10000000; ++i) { p += 10; d += 0.10; fl += 0.10f; }
    std::cout << std::fixed << std::setprecision(6);
    std::cout << "Integer paise total : " << p << " paise = Rs " << p / 100.0 << "\n";
    std::cout << "double total        : Rs " << d << "   error = " << (d - 1000000.0) << "\n";
    std::cout << "float total         : Rs " << fl << "   error = " << (fl - 1000000.0f) << "\n";
```

```
    // Part B: one day, 3,000,000 journeys, mixed fares
    const long long fares[] = {1000, 1200, 2000, 2400, 3000, 3600};
    long long total = 0; double totalRs = 0.0;
    for (long long i = 0; i < 3000000; ++i) {
        long long f = fares[i % 6];
        total += f; totalRs += f / 100.0;
    }
    std::cout << "\n3,000,000 journeys, fares cycle Rs 10/12/20/24/30/36\n";
    std::cout << "long long paise : " << total << " (Rs " << total / 100 << "." << std::setw(2) << std::setfill('0') << total % 100 <<
")\n";
    std::cout << std::setfill(' ');
    std::cout << "double rupees   : " << std::setprecision(6) << totalRs << "\n";
    std::cout << "paise total > INT_MAX ? " << std::boolalpha << (total > INT_MAX) << "  (INT_MAX = " << INT_MAX <<
")\n";


    // Part C: scale table
    std::cout << "\nDaily collection at Rs 30 per journey:\n";
    for (long long n : {20000LL, 500000LL, 3000000LL})
        std::cout << std::setw(8) << n << " journeys -> Rs " << n * 30 << "\n";
}
```

Appendix B – How to reproduce

```
g++ -std=c++17 -Wall -Wextra -o metro metro.cpp && ./metro
g++ -std=c++17 -Wall -Wextra -o sizes sizes.cpp && ./sizes
g++ -std=c++17 -Wall -Wextra -o conv  conv.cpp  && ./conv
g++ -std=c++17 -Wall -Wextra -o ops   ops.cpp   && ./ops
g++ -std=c++17 -Wall -Wextra -o acc   acc.cpp   && ./acc
printf 'abc\n25\n-3\n9\n' | ./input
```

Github link : https://github.com/poojashri07/Object-oriented-c-/tree/main/DSA0103-OOP-CPP-Assigment%201/DSA0103-OOP-CPP